ITP Bleeding Prediction — Updated 9-Month Plan

Revised Timeline, Technical Improvements, Cost Analysis

Last updated: April 2026

---

1. Revised 9-Month Timeline

Phase 1: Foundation and Data Engineering (Months 1–2)

Week	Task	Deliverable
1–2	AWS account setup, IAM roles, VPC configuration, HIPAA compliance review	Secure cloud environment
3–4	Build S3 data lake structure; develop AWS Glue ETL pipeline	Automated data ingestion
5–6	Data cleaning, missing value imputation, exploratory data analysis	Clean dataset + EDA report
7–8	Feature engineering, SMOTE oversampling, train/test split	Feature store in DynamoDB

Nhiên's thesis output: Objective 1 — clinical/lab characteristics + bleeding status description

Phase 2: ML Model Development and Evaluation (Months 3–4)

Week	Task	Deliverable
9–10	Train 4 ML models (RF, XGBoost, LightGBM, LR) in SageMaker Studio	Trained model artifacts
11–12	Hyperparameter optimization via GridSearchCV / Bayesian optimization	Optimized models
13–14	Model evaluation: AUC, sensitivity, specificity, accuracy, calibration curves	Performance comparison report
15–16	SHAP analysis with SageMaker Clarify; feature importance ranking	Explainability report

Nhiên's thesis output: Objective 2 — model comparison + feature importance analysis

Phase 3: Agentic AI Layer (Months 5–6) ⬅️ NEW with extended timeline

Week	Task	Deliverable
17–18	Deploy best ML model to SageMaker endpoint; build Lambda action functions	Real-time inference API
19–20	Chunk and embed clinical guidelines (ASH, ISTH, Vietnamese protocols) into vector store	RAG knowledge base
21–22	Build Bedrock Agents: IDA supervisor, data processing, prediction, explanation agents	Working agent pipeline
23–24	Configure Bedrock Guardrails; implement prompt caching; test agent orchestration	Safe, optimized agent system

Phase 4: Application Development (Month 7) ⬅️ NEW dedicated phase

Week	Task	Deliverable
25–26	Build React web app with Vietnamese UI; integrate API Gateway + Cognito	Frontend application
27–28	Build dashboard: risk gauge, SHAP waterfall chart, explanation panel, patient history	Complete clinical interface

Nhiên's thesis output: Objective 3 — online clinical tool

Phase 5: Testing, Validation, and Optimization (Months 8–9) ⬅️ NEW with extended timeline

Week	Task	Deliverable
29–30	Internal testing: unit tests, integration tests, load testing	Test reports
31–32	User acceptance testing with 5–10 clinicians at BV TMHH	UAT feedback
33–34	A/B testing between models; RAG evaluation using LLM-as-a-judge	Model selection validation
35–36	Cost optimization, documentation, final deployment, thesis writing support	Production system + thesis

2. Technical Improvements (Enabled by 9-Month Timeline)

2.1 Model Improvements

- **Bayesian Hyperparameter Optimization**: Replace GridSearchCV with SageMaker Automatic Model Tuning (Bayesian optimization) for better parameter search across all 4 models
- **Ensemble Stacking**: Build a meta-model that stacks the predictions of RF, XGBoost, LightGBM, and LR for potentially higher AUC than any single model
- **Calibration Curves**: Add Platt scaling or isotonic regression to ensure predicted probabilities are well-calibrated (critical for clinical risk scores)
- **Cross-Validation Strategy**: Use stratified 10-fold CV instead of simple train/test split for more robust evaluation
- **Model Monitoring**: Set up SageMaker Model Monitor to detect data drift and model degradation over time

2.2 Agentic AI Improvements

- **Multi-Turn Conversation**: Enable the IDA agent to handle follow-up questions (e.g., "What if the patient also has infection?") using Bedrock session management
- **LLM-as-a-Judge Evaluation**: Use a second LLM to evaluate the quality, accuracy, and clinical relevance of the Explanation Agent's output — an approach AWS demonstrated for healthcare RAG evaluation
- **Prompt Caching**: Cache the system prompt and clinical guidelines context to reduce latency and cost by up to 90% on repeated queries
- **Intelligent Prompt Routing**: Use Bedrock's routing feature to send simple queries to cheaper models (Nova Micro/Haiku) and complex queries to more capable models (Sonnet)
- **Self-Correcting RAG**: Implement a retrieve-evaluate-refine loop (similar to the Agentic Graph RAG for hepatology paper) where the agent evaluates its own retrieval quality before generating explanations

2.3 Application Improvements

- **QuickSight Dashboard**: Build a population-level analytics dashboard showing bleeding trends, model performance metrics, and feature distribution across all patients
- **A/B Testing Framework**: Deploy two models simultaneously behind the endpoint and compare real-world performance
- **Feedback Loop**: Allow clinicians to rate prediction quality (👍/👎), store feedback in DynamoDB, and use it for future model retraining
- **Offline Mode**: Cache frequently used guideline responses locally so the app works with intermittent internet (common in Vietnamese hospital settings)
- **Mobile Responsive**: Optimize the React app for tablet use at the bedside

2.4 Data Engineering Improvements

- **SageMaker Feature Store**: Replace simple DynamoDB with SageMaker Feature Store for proper feature versioning, lineage tracking, and online/offline serving

- **Data Version Control**: Use S3 versioning + AWS Glue Data Catalog for full data lineage
- **Automated Retraining Pipeline**: SageMaker Pipelines to automatically retrain models when new patient data reaches a threshold

---

3. Similar AWS Architectures and Solutions

3.1 AWS Healthcare Industry Lens — ML Reference Architecture

Source: AWS Well-Architected Healthcare Industry Lens

The official AWS reference architecture for healthcare ML follows this flow:

1. Data collected and pre-processed using a data lake (S3 + Glue)
2. Features extracted and stored in feature stores for model training
3. Ground truth labels populated and reviewed by humans
4. Standard ML training, tuning, and evaluation workflows
5. Models reviewed by cross-functional stakeholders (clinical + regulatory)
6. Accepted models integrated with care delivery IT systems (EHRs)

Alignment with our project: Our architecture follows this reference pattern exactly, with the addition of an Agentic AI layer between the ML model and the end-user interface.

3.2 AWS Guidance for Multi-Modal Data Analysis with Health and ML Services

Source: AWS Solutions Library

This guidance demonstrates how to store, transform, and analyze linked clinical and imaging data, then train ML models for predicting patient outcomes. It uses SageMaker, Athena, and QuickSight.

Alignment: While our project uses structured tabular data (not imaging), the data pipeline pattern (S3 → ETL → Feature Store → SageMaker → QuickSight) is identical.

3.3 ALMA (Advanced Learning Medical Assistant) — CatSalut, Spain

Source: AWS Public Sector Blog (2025)

Catalonia's public health service built an agentic AI assistant using Amazon Bedrock with custom RAG for clinical knowledge management. Results: 65% of 20,000 healthcare professionals integrated it into routine work, with 98% user satisfaction and 98% accuracy.

Alignment: Very similar architecture — Bedrock agents + RAG over clinical guidelines. Validates the feasibility of our Explanation Agent approach.

3.4 NoHarm — Brazil Public Health System

Source: AWS Public Sector Blog (2025)

Deployed in 200+ hospitals, analyzing 5 million prescriptions/month. Uses NER for clinical notes, LLM-based summarization, and feedback-driven learning loops.

Alignment: Demonstrates that LLM-powered clinical decision support works at scale in a developing country's healthcare system — directly relevant to Vietnam.

3.5 AWS Healthcare RAG Evaluation (Radiology)

Source: AWS Machine Learning Blog (2025)

Uses Amazon Bedrock Knowledge Bases evaluation capabilities to assess RAG applications for radiology. Demonstrates LLM-as-a-judge for evaluating clinical AI output quality.

Alignment: We can adopt the same LLM-as-a-judge approach to evaluate our Explanation Agent's output quality and clinical accuracy.

3.6 OMOP CDM on AWS — Clinical Data Platform

Source: DEV Community / AWS Builders (2025)

A reference architecture for building OMOP Common Data Model on AWS, showing S3 → Glue → Feature Store → SageMaker pipeline with governance via Lake Formation and Macie for PHI detection.

Alignment: Our data pipeline follows the same pattern. Could adopt OMOP CDM if BV TMHH standardizes their data model in the future.

---

4. Cost Analysis

Assumptions

- 9-month project duration
- Development and testing usage (not production scale)
- ~50 predictions/day during testing (Months 7–9)
- ~100 RAG queries/day during testing
- 1–2 developers working on the project
- AWS region: ap-southeast-1 (Singapore) — closest to Vietnam

---

Option A: Full Potential Implementation

The maximum-capability architecture with best performance, full features, and production-grade infrastructure.

Monthly Cost Breakdown

Service	Purpose	Monthly Cost	Notes

SageMaker Studio	ML development notebooks	$50–80	ml.m5.xlarge, ~4h/day usage
SageMaker Training	Model training jobs	$30–60	ml.m5.xlarge spot instances, periodic retraining
SageMaker Endpoint	Real-time ML inference	$80–120	ml.m5.large always-on endpoint
SageMaker Clarify	SHAP explanations	$20–30	Per-inference SHAP computation
SageMaker Feature Store	Feature versioning + serving	$15–25	Online + offline store
SageMaker Model Monitor	Drift detection	$10–15	Continuous monitoring
Bedrock (Claude Sonnet)	LLM for explanation agent	$60–120	~100 queries/day, ~3K input + 500 output tokens each
Bedrock Agents	Agent orchestration	$20–40	Token overhead for agent reasoning
Bedrock Knowledge Base	RAG pipeline	$10–15	Embedding + retrieval
Bedrock Guardrails	Safety filtering	$10–20	Per-request filtering
OpenSearch Serverless	Vector store for RAG	$350–400	**Minimum floor: ~$350/month (0.5 OCU)**
Amazon S3	Data lake	$5–10	Storage + requests
AWS Glue	ETL pipeline	$10–20	Crawler + ETL job runs
DynamoDB	Patient data + feedback	$5–10	On-demand capacity
Lambda	Agent action functions	$2–5	Pay per invocation
API Gateway	REST API	$5–10	Per-request pricing
Cognito	Authentication	$0–5	Free tier covers <50K MAU
Amplify/CloudFront	Web hosting + CDN	$5–10	Low traffic during research
CloudWatch	Monitoring + logging	$10–20	Logs, metrics, alarms
QuickSight	Analytics dashboard	$24	1 author license

AWS Budgets	Cost alerts	$0	Free

Option A Monthly Total: $720–1,040/month

Option A 9-Month Total: $6,500–$9,400

AWS Services Used (Option A): 20 services

1. Amazon S3
2. AWS Glue (ETL + Data Catalog)
3. Amazon DynamoDB
4. Amazon SageMaker Studio
5. Amazon SageMaker Training
6. Amazon SageMaker Endpoint (real-time)
7. Amazon SageMaker Clarify
8. Amazon SageMaker Feature Store
9. Amazon SageMaker Model Monitor
10. Amazon Bedrock (Claude Sonnet / Haiku)
11. Amazon Bedrock Agents
12. Amazon Bedrock Knowledge Bases
13. Amazon Bedrock Guardrails
14. Amazon OpenSearch Serverless
15. AWS Lambda
16. Amazon API Gateway
17. Amazon Cognito
18. AWS Amplify / Amazon CloudFront
19. Amazon CloudWatch
20. Amazon QuickSight

Option B: Cost-Optimized Implementation

Achieves the same core research goals with significant cost reductions. Trades some production-grade features for affordability.

Key Cost Optimization Strategies

1. **Replace OpenSearch Serverless with pgvector on Aurora Serverless v2** — eliminates the $350/month floor
2. **Use SageMaker Serverless Inference** instead of always-on endpoints — pay only per request
3. **Use Amazon Nova Micro / Claude Haiku** instead of Sonnet — 10–50x cheaper per token
4. **Use Intelligent Prompt Routing** — auto-route simple queries to cheapest model

5. **Enable Prompt Caching** — cache system prompts and guideline context (up to 90% savings)
6. **Use Batch inference** for non-real-time tasks (50% discount)
7. **Replace SageMaker Studio with local Jupyter** — train locally, deploy on AWS
8. **Use S3 + pandas instead of SageMaker Feature Store** — simpler, free
9. **Skip QuickSight** — use built-in React charts instead
10. **Use SageMaker Model Monitor only monthly** — not continuous

Monthly Cost Breakdown

Service	Purpose	Monthly Cost	Notes
Local Jupyter + S3	ML development	$0	Train locally, upload artifacts to S3
SageMaker Training	Model training jobs	$10–20	Spot instances, infrequent runs
SageMaker Serverless Inference	ML inference	$5–15	Pay per request, scales to zero
Bedrock (Nova Micro + Haiku)	LLM for explanation agent	$5–15	Nova Micro: $0.035/1Minput, Haiku:$ 0.80/1M input
Bedrock Agents	Agent orchestration	$5–10	Cheaper with Haiku backbone
Bedrock Knowledge Base	RAG pipeline	$5–10	Embedding costs only
Bedrock Prompt Caching	Reduce repeated context cost	-5to—10	Savings on cached guideline context
Aurora Serverless v2 (pgvector)	Vector store for RAG	$15–30	Scales to 0.5 ACU minimum (~$0.12/ACU-hour)
Amazon S3	Data lake + feature store	$3–5	Also used as simple feature store
AWS Glue	ETL pipeline	$5–10	Minimal jobs
DynamoDB	Patient data + feedback	$2–5	On-demand, low volume
Lambda	Agent action functions	$1–3	Free tier: 1M requests/month
API Gateway	REST API	$2–5	Low volume
Cognito	Authentication	$0	Free tier

S3 + CloudFront	Web hosting	$2–5	Static site hosting
CloudWatch	Monitoring (basic)	$5–10	Essential metrics only

Option B Monthly Total: $65–155/month

Option B 9-Month Total: $585–1,395

AWS Services Used (Option B): 14 services

1. Amazon S3
2. AWS Glue
3. Amazon DynamoDB
4. Amazon SageMaker Training (spot instances)
5. Amazon SageMaker Serverless Inference
6. Amazon Bedrock (Nova Micro + Claude Haiku)
7. Amazon Bedrock Agents
8. Amazon Bedrock Knowledge Bases
9. Amazon Aurora Serverless v2 (pgvector)
10. AWS Lambda
11. Amazon API Gateway
12. Amazon Cognito
13. Amazon CloudFront
14. Amazon CloudWatch

---

5. Cost Comparison Summary

Metric	Option A: Full Potential	Option B: Cost-Optimized
Monthly cost	$720–1,040	$65–155
9-month total	$6,500–9,400	$585–1,395
AWS services	20	14
LLM model	Claude Sonnet (best quality)	Nova Micro / Haiku (good quality)
Vector store	OpenSearch Serverless ($350 floor)	Aurora pgvector (scales to ~$15)
ML inference	Always-on endpoint	Serverless (pay per request)
Monitoring	Continuous (Model Monitor + CloudWatch)	Basic (monthly checks)

Dashboard	QuickSight (professional)	React charts (built-in)
Feature store	SageMaker Feature Store	S3 + pandas (simple)
Explanation quality	High (Sonnet)	Good (Haiku) — adequate for research
Cold start latency	None (always-on)	1–3 seconds (serverless)
Best for	Publication-ready, production-grade	Research project, thesis demonstration

Recommendation

Start with Option B during Months 1–7 (development and initial testing), then **selectively upgrade to Option A components** in Months 8–9 if budget allows and specific features are needed. The critical upgrade path:

1. First upgrade: Aurora pgvector → OpenSearch Serverless (only if RAG quality is insufficient)
2. Second upgrade: Haiku → Sonnet (only if explanation quality needs improvement)
3. Third upgrade: Serverless Inference → always-on endpoint (only if latency is a problem during UAT)

This staged approach keeps the 9-month total at approximately **$800–2,500** while maintaining the flexibility to scale up where it matters.

---

6. Cost Optimization Tips

- **AWS Activate for Startups/Research**: Apply for AWS credits (up to $5,000$–100,000) — academic/research projects often qualify
- **AWS Free Tier**: Lambda (1M requests), DynamoDB (25GB), S3 (5GB), Cognito (50K MAU), CloudWatch (basic) are free
- **Spot Instances**: Use for SageMaker training — 60–90% savings over on-demand
- **Bedrock Flex Tier**: 50% discount for non-urgent inference (same API, asynchronous processing)
- **Prompt Caching**: Cache the ASH/ISTH guidelines system prompt — up to 90% savings on repeated context
- **AWS Budgets**: Set hard monthly limits and alerts at 50%, 80%, 100% thresholds
- **Turn off what you don't use**: Delete SageMaker endpoints and notebook instances when not in use — idle endpoints are the #1 cause of unexpected AWS ML bills
- **Region selection**: ap-southeast-1 (Singapore) is closest to Vietnam but slightly more expensive than us-east-1; consider us-east-1 if data residency is not a concern

---

7. Risk Mitigation

Risk	Impact	Mitigation
OpenSearch Serverless cost floor ($350/month)	Budget overrun	Use Aurora pgvector (Option B)
Bedrock agent token multiplication (5–10x)	Unexpected LLM costs	Set MaxTokens limits; use prompt caching
Idle SageMaker endpoint running 24/7	$80–120/month wasted	Use Serverless Inference or auto-shutdown schedule
CloudWatch log volume	$0.50/GB ingestion adds up	Set log retention to 7 days; filter verbose logs
LLM hallucination in clinical context	Patient safety	Bedrock Guardrails + structured output templates
Small dataset (n~150) overfitting	Poor generalization	Cross-validation, regularization, ensemble methods
Data quality from hospital EHR	Noisy features	Extensive EDA in Phase 1; clinical validation by Nhiên

---

8. Final Architecture Decision Matrix

Decision	Option A Choice	Option B Choice	Recommendation
ML training environment	SageMaker Studio	Local Jupyter	Start local, move to SageMaker if needed
ML inference	SageMaker Endpoint (always-on)	SageMaker Serverless	Serverless for research
LLM model	Claude Sonnet	Nova Micro + Haiku	Haiku (best quality/cost balance)
Vector database	OpenSearch Serverless	Aurora pgvector	Aurora pgvector
Feature store	SageMaker Feature Store	S3 + DynamoDB	S3 + DynamoDB
Monitoring	Model Monitor + CloudWatch	CloudWatch basic	CloudWatch basic
Dashboard	QuickSight	React built-in charts	React charts

Web hosting	Amplify	S3 + CloudFront	S3 + CloudFront

This updated plan extends the original 6-month timeline to 9 months, adding dedicated phases for advanced testing, clinical validation, and cost optimization. The hybrid approach (ML core + Agentic AI wrapper) remains the recommended strategy.